UNIT - I

INTRODUCTION TO DATA STRUCTURES AND ALGORITHMS

CO1: Understand and Analyze various types of data structures and algorithms.

1.1 INTRODUCTION TO DATA STRUCTURE

Data Structure is a way of collecting and organizing data in such a way that we can perform operations on these data in an effective way. Data Structures is about rendering data elements in terms of some relationship, for better organization and storage. For example, we have some data which has, player's name "Anil" and age 26. Here "Anil" is of String data type and 26 is of integer data type.

We can organize this data as a record like Player record, which will have both player's name and age in it. Now we can collect and store player's records in a file or database as a data structure.

Example: "Nishant" 30, "Chetan" 31, "Sachin" 33

If you are aware of Object Oriented programming concepts, then a class also does the same thing, it collects different type of data under one single entity. The only difference being, is that data structures provide for techniques to access and manipulate data efficiently.

In simple language, Data Structures are structures programmed to store ordered data, so that various operations can be performed on it easily. It represents the knowledge of data to be organized in memory. It should be designed and implemented in such a way that it reduces the complexity and increases the efficiency.

Basic Types of Data Structures

As we have discussed above, anything that can store data can be called as a data structure, hence Integer, Float, Boolean, Char etc., all are data structures. They are known as Primitive Data Structures.

Then we also have some complex Data Structures, which are used to store large and connected data.

Some examples of Abstract Data Structure are:

- Linked List
- Tree
- Graph
- Stack, Queue etc.

All these data structures allow us to perform different operations on data. We select these data structures based on which type of operation is required. We will look into these data structures in more details in our later units.

1.2 DATA, INFORMATION AND KNOWLEDGE

Data:

Data is a raw and unorganized fact that required to be processed to make it meaningful. It can be considered as a facts and statistics collected together for reference or analysis.

Data are individual units of information. In analytical processes, data are represented by variables. Data is always interpreted, by a human or machine, to derive meaning. So, data is meaningless. Data contains numbers, statements and characters in a raw form.

Information:

Information is delineated because the structured, organized and processed data, conferred inside context, that makes it relevant and helpful to the one who desires it. Data suggests that raw facts and figures regarding individuals, places or the other issue, that is expressed within the type of numbers, letters or symbols.

Information is the knowledge that is remodeled and classified into an intelligible type, which may be utilized in the method of deciding. In short, once knowledge end up to be purposeful when conversion, it is referred to as information. It is one thing that informs, in essence, it provides a solution to a specific question. It may be obtained from numerous sources like newspaper, internet, television, people, books etc.

Table 1.1: Difference between Information and Data

Sr. No.	Data	Information
1.	Data are the variables which helps to develop ideas/conclusions.	Information is the meaningful of data.
2.	Data are text and numerical values.	Information is refined form of actual data.
3.	Data does not rely on Information.	While Information relies on Data.
4.	Bits and Bytes are the measuring unit of data.	Information is measured in meaningful units like time, quantity, etc.
5.	Data can be easily structured as the following: 1. Tabular data 2. Graph 3. Data tree	Information can also be structured as the following: 1. Language 2. Ideas 3. Thoughts

Sr. No.	Data	Information
6.	Data does not have any specific purpose.	Information carries a meaning that has been assigned by interpreting data.
7.	It is low-level knowledge.	It is the second level of knowledge.
8.	Data does not directly help in decision making.	Information directly helps in decision making.

Knowledge:

Rules, algorithms, interpreters (such as pattern recognizers) or other mechanisms, including those that exist in the human brain (regardless of our ability to describe those mechanisms) that transform data into information. Knowledge may be changed by its interaction with information.

1.2.1 Data Object

A data object is a collection of one or more data points that create meaning as a whole. In other words, "data object" is an alternate way of saying "this group of data should be thought of as standalone."

The most common example of a data object is a data table, but others include arrays, pointers, records, files, sets and scalar types.

Values in a data object may have their own unique IDs, data types and attributes. In this way, data objects vary across database structures and different programming languages.

An easy way to think about the term "data object" is that it reflects the simple need for compartmentalizing information in a database that otherwise looks big and confusing.

As a list, data objects consist of:

- **Values:** The data itself.
- **Unique IDs:** One data point that identifies others related to it.
- **Attributes:** Additional data within one Unique ID.
- **Data Types:** Classifications of data such as text, numeric and boolean.

Example of Data Objects in Databases

Imagine you run an e-commerce company that sells watches to retailers. Your company is called Batch Watch and its key units are vendors, products and customers. Over time, you have built a complex database with a large amount of data.

As a raw database, your data looks like an enormous table. It is nearly impossible to conceptualize or extract insight from the data in that format. Instead, you hire an external consultant to build a data model that helps you understand. What is in your data model? You guessed it: data objects.

These data objects are subsets of your huge database. For example, while the huge database contains vendor, product and customer data, a data object might only contain vendor data. Another contains product data. And still another, customer data.

As you might imagine, these data objects can still contain an inconceivable amount of data. That is why data objects actually have 2 "views," or structures: the underlying data table and a box-style view. Let us explore these briefly:

1. Underlying Data Tables

These are data tables made of columns and rows. Here is a very simple example of a vendor data Table 1.2 with a unique ID and 3 attributes:

Table 1.2: Vendor Data Table

ID	Att 1	Att 2	Att 3
Vendor ID	Avg. Sale	Lat Purchase	# items purchased
MetalOne Inc.	100	9/1/20	3
GlassHouse Inc.	145	6/12/20	4

While our example is here is simplified, the data object can still be quite cumbersome to understand. Who can put layers or rows and columns into context *easily? In my* experience, not many. That is why there is the second view: the box-style data object view.

2. Box-Style View

The box style view displays only the column headers in a data table to summarize the object. Here is a box-style view of the data table above that uses metadata as a summary:

Table 1.3: Box View of Data Object

Vendors
Vendor ID <<PK>>
Attributes: Avg. sale
Last Purchase
of items purchased

The <<PK>> reference indicates *"primary key," which in* another term for unique ID. With the box-style view of the data object, it is easy to understand what is inside the data object at a glance. This quick-information view is data object is added value.

1.2.2 Types of Data Objects in Databases

We have talked exclusively about data tables up to this point, but they are not the only type of data object. Here is a list of additional data objects that analysts encounter while working in databases:

1. Arrays
2. Pointers
3. Records
4. Files
5. Sets
6. Scalar types

1. **Arrays:** Arrays are data objects that have only one dimension. You can think of them as a single column in a data table. Analysts use arrays when their data is not thorough enough to merit a table.
2. **Pointers:** A data object pointer is a special data value that indicates the memory location of another data point or group of data points. In most cases, a pointer is included within a table column as a separate dimension.

3. **Records:** Records are data objects that contain one data entry for each dimension in the data table. You can think of data records as a single row in a data table. Analysts use records any time they enter an observation, which usually have values for each dimension in the table.
4. **Files:** Data object files use code to ensure other data objects use the right structure. While they are not executable like the other data objects, data object files are invaluable. They ensure data integrity by ensuring data entries take on the same format. An easy way to conceptualize files is that unlike other data objects, you cannot view them in simple database software like Excel.
5. **Sets:** Data object sets combine multiple data objects and, in most cases, tables. In this way, data sets occupy a level of hierarchy between data tables and databases. In common speak, analysts use the shortened term "data set" to refer to data object sets. In addition, this term has become a default term for all data objects.
6. **Scalar Types:** A scalar data object indicates a single value rather than an aggregation like data tables, arrays and records. That said, the term "scalar data" takes on different meanings depending on the data.

1.3 DATA STRUCTURE

Data Structure is a way to organized data in such a way that it can be used efficiently. It is a programmatic way of storing data so that data can be used efficiently. Data structures can implement one or more particular Abstract Data Types (ADT), which specify the operations that can be performed on a data structure and the computational complexity of those operations. In comparison, a data structure is a concrete implementation of the specification provided by an ADT.

Characteristics of a Data Structure:

- **Correctness:** Data Structure implementation should implement its interface correctly.
- **Time Complexity:** Running time or execution time of operations of data structure must be as small as possible.
- **Space Complexity:** Memory usage of a data structure operation should be as little as possible.

Need for Data Structure

As applications are getting complex and data rich, there are three common problems applications face now-a-days.

1. **Data Search:** Consider an inventory of 1 million (10^6) items of a store. If application is to search an item. It has to search item in 1 million (10^6) items every time slowing down the search. As data grows, search will become slower.
2. **Processor Speed:** Processor speed although being very high, falls limited if data grows to billion records.
3. **Multiple Requests:** As thousands of users can search data simultaneously on a web server, even very fast server fails while searching the data.

To solve above problems, data structures come to rescue. Data can be organized in a data structure in such a way that all items may not be required to be search and required data can be searched almost instantly.

1.4 ABSTRACT DATA TYPE (ADT)

An Abstract Data Type (ADT) is defined as a data declaration packaged together with the operations that are meaningful for the data type. In other words, we encapsulate the data and the operations and we hide them from a user.

The Concept of Abstraction Means:

1. We know what the data type can do?
2. How it is done, is hidden?

Consider the concept of a list. At least three data structures will support a list. We can use an array, a linked list or a file. If we place our list in an ADT (Abstract Data Type), users should not be aware of the structure we use. As long as they can insert and retrieve data, it should make no difference how we store the data.

As another example, consider the office incharge who needs to simulate the waiting line at an exam section counter. He needs to determine how many windows are needed to attend students efficiently. This analysis requires the simulation of a queue.

However, queues are not generally available in programming languages. Even if queue types were available, we would still need some basic queue operations such as enqueuing (insertion) and dequeuing (deleting) for a simulation.

Abstract data types are the special type of data type whose representation is hidden from the user. In the concept of abstraction user is aware about the types and use of data type but user is completely unaware about the how data type can perform the operation.

In abstract data type data and operation is encapsulated into the single unit. Abstract data type does not allow user to see how the implementation works.

Abstract data type provides the more convenient way to organize large amount of data. With the help of abstract data type user can set the different levels of abstraction to the data. Abstract data type stores the data along with the operations that to be performed on that data and the error condition associate with the operations. Same abstract data type can be used by different user.

1.5 CLASSIFICATION OF DATA STRUCTURES (LINEAR AND NON-LINEAR) [Oct. 22]

Data structure can be classified in several ways. As per the definition of data structure that it is the logical way of storing data so it can be classified according to way it stores the data. Initially data structure can only be classified into two types i.e. primitive and non-primitive. As the new technologies are evolving day by day so non primitive data structure can be classified in the following Fig. 1.1.

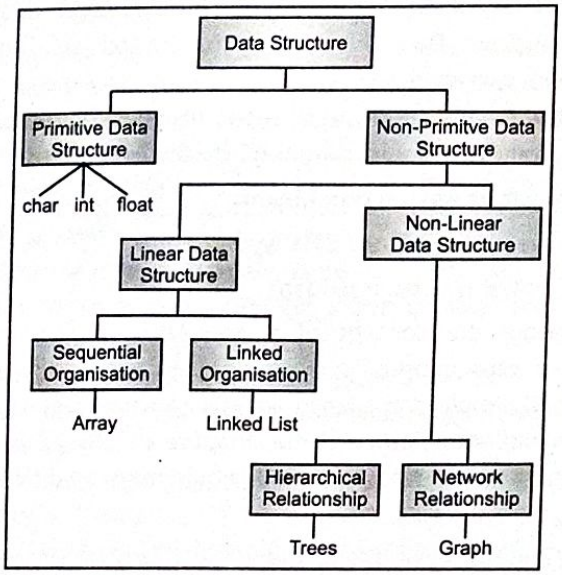


Fig. 1.1: Data Structure classification

1.5.1 Primitive Data Structure

Data structures which are directly operated by machine-level instructions are known as primitive data structures. This type of data structure does not involve other type of elements. E.g. Data structure defines for handling integer or character type of values. Primitive data structure are the fundamental data structures and can be built by using primitive data types such as int, char, float.

1.5.2 Non-Primitive Data Structure

Non-primitive data structures are more complex as compare to primitive data structure. This type of data structure is more advanced but derived from primitive data structure. E.g. array is the non-primitive data structure which is used to store similar type of elements.

1.5.3 Linear Data Structure

It is the non-primitive data structure in which all the elements are arranged in the form of sequence or list. This type of data structure possesses linear relationship among the data members. Data members of linear data structures have unique predecessor and unique successor.

Linear data structure can be classified into following types according to the way data organize in data structure:

- **Sequential Data Structure:** In sequential data structure elements are arranged in the one after another fashion and their sequence is maintained. Array is the example of sequential organization where similar types of elements are stored in consecutive memory location.

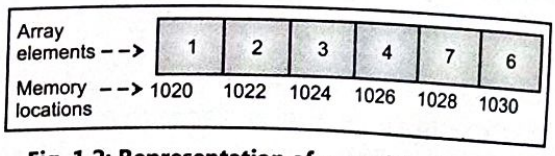


Fig. 1.2: Representation of array data structure

- **Linked Data Structure:** In linked data structure elements are arranged one after the other fashion but memory is dynamically allocated. Relationship among these elements is maintained by means of pointer. Linked list is the example of linked data structure.

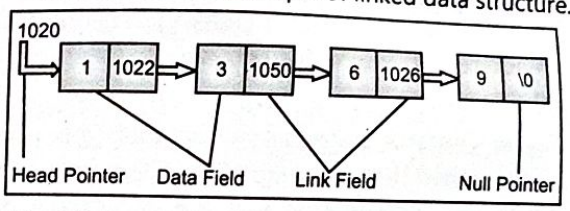


Fig. 1.3: Representation of linked list data structure

1.5.4 Non-Linear Data Structure

It is the non-primitive data structure in which all the elements are arranged in the form of hierarchical or network fashion. This type of data structure does not possess linear relationship among the data members. Data members of linear data structures have one or more predecessor and successor. Non-linear data structure can be classified into following types according to the relationship among the data members.

- **Hierarchical Data Structure:** In hierarchical data structure elements are stored one after the other but their hierarchy is maintained. Tree is the example of hierarchical data structure where elements are stored and their hierarchy is maintain as per specific format.

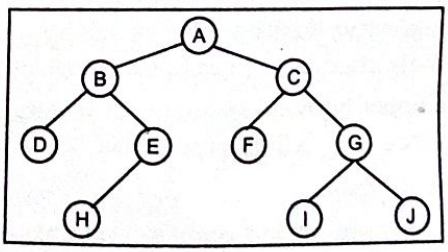


Fig. 1.4: Representation of tree data structure

- **Network Data Structure:** In network data structure elements are stored one after the other but their relation in a defined format. Graph is the example of network data structure in which data elements are stored as per their defined relation.

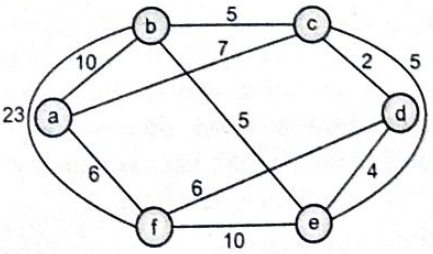


Fig. 1.5: Representation of graph data structure

It is the non-primitive data structure in which all the elements are arranged in the form of hierarchical or network fashion. This type of data structure does not possess linear relationship among the data members.

1.5.5 Static Data Structure

In static data structure numbers of elements are known at the initial level. Static data structure is also called as fixed size data structure. Static data structure is created before the program execution begins. Array is the example of static data structure. Before defining the array, user must know the size of array. If the defined size of array is less than the required size then extra elements are not consider where as if defined size is greater than the required size then extra allocated memory is wasted. To avoid these problems there is a need of dynamic data structure.

1.5.6 Dynamic Data Structure

In dynamic data structure, memory allocation is performed at the run time. So, there is no need to collect information about the memory size at the initial level. Dynamic data structure is created at run time. Linked list is the example of dynamic data structure. In linked list new node is created at the time of inserting element to linked list. The problem of memory wastage is overcome in linked list but in linked list accessing element is difficult as compare to array.

Table 1.4: Difference Between Static and Dynamic Data Structure

Sr. No	Static Data Structure	Dynamic Data Structure
1.	It is fixed size data structure	It is flexible size data structure
2.	Faster access to elements	Slower access to elements
3.	It is difficult to add, remove or modify elements.	Elements can be efficiently add, remove or modify.
4.	Less effective utilization of resources	Effective utilization of resources
5.	E.g. Arrays	E.g. Linked list

1.5.7 Persistent Data Structure

Data structure stores the data in efficient way so that user can perform several operations on the data. During operation data is modified and new version of database creates. The data structure which maintains both the versions of data structure is known as persistent data structure. Functional data structure is the example of persistent data structure. Persistent data structure is of two types:

1. **Partially Persistent Data Structure:** In this data structure if modification is done on data structure, then only recent version is update.
2. **Fully Persistent Data Structure:** In this data structure if modification is done on data structure, then both the versions are updated.

1.5.8 Ephemeral Data Structure

The data structure which supports modification operation only on recent copy is known as Ephemeral data structure. Ephemeral data structure follows conventional imperative programming paradigms. Ephemeral data structure is change over time if any modification is performed on the content. This type of data structure is used when resources are shared among several clients. Imperative data structure is the example of ephemeral data structure. Ephemeral data structure is more complex than persistent data structure.

1.6 COMPLEXITY OF ALGORITHM: SPACE COMPLEXITY, TIME COMPLEXITY [Oct. 22]

Complexity of an algorithm is a measure of the amount of time and/or space required by an algorithm for an input of a given size (n).

What Effects Run Time of an Algorithm?

1. Computer used, the hardware platform
2. Representation of Abstract Data Types (ADT's)
3. Efficiency of compiler
4. Competence of implementer (programming skills)
5. Complexity of underlying algorithm
6. Size of the input

We will show that of those above (5) and (6) are generally the most significant.

Time for an Algorithm to Run t(n)

It is a function of an input. However, we will attempt to characterize this by the size of the input. We will try and estimate the worst case and sometimes the best case and very rarely the average case.

What Do We Measure?

In analyzing an algorithm, rather than a piece of code, we will try and predict the number of times "the principal activity" of that algorithm is performed. For example, if we are analyzing a sorting algorithm, we might count the number of comparisons performed and if it is an algorithm to find some

optimal solution, the number of times it evaluates a solution. If it is a graph coloring algorithm, we might count the number of times we check that a colored node is compatible with its neighbors.

1. Worst Case

It is the maximum run time, over all inputs of size n, ignoring effects (a) through (d) above. That is, we only consider the "number of times the principal activity of that algorithm is performed".

2. Best Case

In this case we look at specific instances of input of size n. For example, we might get best behaviour from a sorting algorithm if the input to it is already sorted.

3. Average Case

Arguably, average case is the most useful measure. It might be the case that worst case behavior is pathological and extremely rare and that we are more concerned about how the algorithm runs in the general case. Unfortunately, this is typically a very difficult thing to measure. Firstly, we must in some way be able to define by what we mean as the "average input of size n". We would need to know a great deal about the distribution of cases throughout all data sets of size n. Alternatively we might make a possibly dangerous assumption that all data sets of size n are equally likely. Generally, in order to get a feel for the average case we must resort to an empirical study of the algorithm and in some way classify the input (and it is only recently with the advent of high-performance, low-cost computation, that we can seriously consider this option).

We cannot talk about Efficiency of Algorithms and Data Structures without explaining the term "algorithm complexity", which we have already mentioned several times in one form or another. We will avoid the mathematical definitions and we are going to give a simple explanation of what the term means.

Algorithm Complexity is a Measure which evaluates the order of the Count of Operations, performed by a given or algorithm as a function of the size of the input data. To put this simpler, complexity is a rough Approximation of the Number of Steps necessary to execute an algorithm. When we evaluate complexity we speak of order of operation count, not of their exact count. For example, if we have an order of N^2 operations to process N elements, then $N^2/2$ and $3N^2$ are of one and the same quadratic order.

Algorithm complexity is commonly represented with the O(f) notation, also known as asymptotic notation or "Big O notation", where f is the function of the size of the input data. The asymptotic computational complexity O(f) measures the order of the consumed resources (CPU time, memory, etc.) by certain algorithm expressed as function of the input data size.

Complexity can be constant, logarithmic, linear, $n \times \log(n)$, quadratic, cubic, exponential, etc. This is respectively the order of constant, logarithmic, linear and so on, number of steps, are executed to solve a given problem. For simplicity, sometime instead of "algorithms complexity" or just "complexity" we use the term "running time".

Typical Algorithm Complexities

This Table 1.5 will explain what every type of complexity (running time) means:

Table 1.5

Complexity	Running Time	Description
constant	O(1)	It takes a constant number of steps for performing a given operation (for example 1, 5, 10 or other number) and this count does not depend on the size of the input data.
logarithmic	O(log(N))	It takes the order of log(N) steps, where the base of the logarithm is most often 2, for performing a given operation on N elements. For example, if N = 1,000,000, an algorithm with a complexity O(log(N)) would do about 20 steps (with a constant precision). Since the base of the logarithm is not of a vital importance for the order of the operation count, it is usually omitted.
linear	O(N)	It takes nearly the same number of steps as the number of elements for performing an operation on N elements. For example, if we have 1,000 elements, it takes about 1,000 steps. Linear complexity means that the number of elements and the number of steps are linearly dependent, for example the number of steps for N elements can be N/2 or 3N.
	O(nlog(n))	It takes Nlog(N) steps for performing a given operation on N elements. For example, if you have 1,000 elements, it will take about 10,000 steps.
quadratic	$O(n^2)$	It takes the order of N^2 number of steps, where the N is the size of the input data, for performing a given operation. For example, if N = 100, it takes about 10,000 steps. Actually, we have a quadratic complexity when the number of steps is in quadratic relation with the size of the input data. For example, for N elements the steps can be of the order of $3N^2/2$.

Complexity	Running Time	Description
cubic	$O(n^3)$	It takes the order of N^3 steps, where N is the size of the input data, for performing an operation on N elements. For example, if we have 100 elements, it takes about 1,000,000 steps.
exponential	$O(2^n)$, $O(N!)$, $O(n^k)$, ...	It takes a number of steps, which is with an exponential dependability with the size of the input data, to perform an operation on N elements. For example, if N = 10, the exponential function 2^N has a value of 1024, if N = 20, it has a value of 1048 576 and if N = 100, it has a value of a number with about 30 digits. The exponential function N! grows even faster: for N = 5 it has a value of 120, for N = 10 it has a value of 3,628,800 and for N = 20 – 2,432,90,008,176,640,000.

When evaluating complexity, constants are not taken into account, because they do not significantly affect the count of operations. Therefore, an algorithm which does N steps and algorithms which do N/2 or 3N respectively are considered linear and approximately equally efficient, because they perform a number of operations which is of the same order.

Complexity and Execution Time

The execution speed of a program depends on the complexity of the algorithm, which is executed. If this complexity is low, the program will execute fast even for a big number of elements. If the complexity is high, the program will execute slowly or will not even work (it will hang) for a big number of elements.

If we take an average computer from 2008, we can assume that it can perform about 50,000,000 elementary operations per second. This number is a rough approximation, of course. The different processors work with a different speed and the different elementary operations are performed with a different speed and also the computer technology constantly evolves. Still, if we accept, we use an average home computer from 2008, we can make the following conclusions about the speed of execution of a given program depending on the algorithm complexity and size of the input data.

Table 1.6

Algorithm	10	20	50	100	1,000	10,000	100,000
O(1)	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.
O(log(n))	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.
O(n)	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.
O(nlog(n))	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.
$O(n^2)$	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	2 sec.	3-4 min.
$O(n^3)$	< 1 sec.	< 1 sec.	< 1 sec.	< 1 sec.	20 sec.	5.55 hours	231.5 days
$O(2^n)$	< 1 sec.	< 1 sec.	260 days	hangs	hangs	hangs	hangs
O(n!)	< 1 sec.	hangs	hangs	hangs	hangs	hangs	hangs
$O(n^n)$	3-4 min.	hangs	hangs	hangs	hangs	hangs	hangs

We can draw many conclusions from the above table:

- Algorithms with a constant, logarithmic or linear complexity are so fast that we cannot feel any delay, even with a relatively big size of the input data.
- Complexity O(nlog(n)) is similar to the linear and works nearly as fast as linear, so it will be very difficult to feel any delay.
- Quadratic algorithms work very well up to several thousand elements.
- Cubic algorithms work well if the elements are not more than 1,000.
- Generally, these called polynomial algorithms (any, which are not exponential) are considered to be fast and working well for thousands of elements.
- Generally, the exponential algorithms do not work well and we should avoid them (when possible). If we have an exponential solution to a task, maybe we actually do not have a solution, because it will work only if the number of the elements is below 10-20. Modern cryptography is based exactly on this - there are not any fast (non-exponential) algorithms for finding the secret keys used for data encryption.

1.6.1 Time Complexity

For any defined problem, there can be N number of solutions. This is true in general. If I have a problem and I discuss about the problem with all of my friends, they will all suggest me different solutions. And I am the one who has to decide which solution is the best based on the circumstances.

Similarly for any problem which must be solved using a program, there can be infinite number of solutions. Let's take a simple example to understand this. Below we have two different algorithms to find square of a number (for some time, forget that square of any number n is $n \times n$).

One solution to this problem can be, running a loop for n times, starting with the number n and adding n to it, every time.

```
/*
we have to calculate the square of n
*/
for i=1 to n
do n = n + n
// when the loop ends n will hold its square
return n
```

Or, we can simply use a mathematical operator * to find the square.

```
/*
we have to calculate the square of n
*/
return n*n
```

In the above two simple algorithms, you saw how a single problem can have many solutions. While the first solution required a loop which will execute for n number of times, the second solution used a mathematical operator * to return the result in one line. So which one is the better approach, of course the second one.

Time complexity of an algorithm signifies the total time required by the program to run till its completion.

The time complexity of algorithms is most commonly expressed using the big O notation. It's an asymptotic notation to represent the time complexity.

Time Complexity is most commonly estimated by counting the number of elementary steps performed by any algorithm to finish execution. Like in the example above, for the first code the loop will run n number of times, so the time complexity will be n atleast and as the value of n will increase the time taken will also increase. While for the second code, time complexity is constant, because it will never be dependent on the value of n, it will always give the result in 1 step.

And since the algorithm's performance may vary with different types of input data, hence for an algorithm we usually use the worst-case time complexity of an algorithm because that is the maximum time taken for any input size.

Calculating Time Complexity

Now let's tap onto the next big topic related to Time complexity, which is How to Calculate Time Complexity. It becomes very confusing some times, but we will try to explain it in the simplest way.

Now the most common metric for calculating time complexity is Big O notation. This removes all constant factors so that the running time can be estimated in relation to N, as N approaches infinity. In general, you can think of it like this:

Statement

Above we have a single statement. Its Time Complexity will be Constant. The running time of the statement will not change in relation to N.

```
for i in range(N):
  statement
```

The time complexity for the above algorithm will be Linear. The running time of the loop is directly proportional to N. When N doubles, so does the running time.

```
for i in range(N):
  for j in range(N):
    statement
```

This time, the time complexity for the above code will be Quadratic. The running time of the two loops is proportional to the square of N. When N doubles, the running time increases by N × N.

```
while low <= high:
  mid = (low + high) // 2
  if target < list[mid]:
    high = mid - 1
  elif target > list[mid]:
    low = mid + 1
  else:
    break
```

This is an algorithm to break a set of numbers into halves, to search a particular field (we will study this in detail later). Now, this algorithm will have a Logarithmic Time Complexity. The running time of the algorithm is proportional to the number of times N can be divided by 2(N is high-low here). This is because the algorithm divides the working area in half with each iteration.

```
def quicksort(arr, left, right):
  if left < right:
    pivot = partition(arr, left, right)
    quicksort(arr, left, pivot - 1)
    quicksort(arr, pivot + 1, right)
```

Taking the previous algorithm forward, above we have a small logic of Quick Sort. Now in Quick Sort, we divide the list into halves every time, but we repeat the iteration N times (where N is the size of list). Hence time complexity will be Nlog(N). The running time consists of N loops (iterative or recursive) that are logarithmic, thus the algorithm is a combination of linear and logarithmic.

Types of Notations for Time Complexity

Now we will discuss and understand the various notations used for Time Complexity.

1. **Big Oh** denotes "fewer than or the same as" <expression> iterations.
2. **Big Omega** denotes "more than or the same as" <expression> iterations.
3. **Big Theta** denotes "the same as" <expression> iterations.
4. **Little Oh** denotes "fewer than" <expression> iterations.
5. **Little Omega** denotes "more than" <expression> iterations.

Understanding Notations of Time Complexity with Example

- **O(Expression)** is the set of functions that grow slower than or at the same rate as expression. It indicates the maximum required by an algorithm for all input values. It represents the worst case of an algorithm's time complexity.
- **Omega(Expression)** is the set of functions that grow faster than or at the same rate as expression. It indicates the minimum time required by an algorithm for all input values. It represents the best case of an algorithm's time complexity.
- **Theta(Expression)** consists of all the functions that lie in both O(expression) and Omega(expression). It indicates the average bound of an algorithm. It represents the average case of an algorithm's time complexity.

Suppose you have calculated that an algorithm takes f(n) operations, were,

```
f(n)=3*n^2+2*n +4. // n^2 means square of n
```

Since this polynomial grows at the same rate as n^2, then you could say that the function f lies in the set Theta(n^2). (It also lies in the sets O(n^2) and Omega(n^2) for the same reason.)

The simplest explanation is, because Theta denotes the same as the expression. Hence, as f(n) grows by a factor of n^2, the time complexity can be best represented as Theta(n^2).

1.6.2 Space Complexity

Whenever a solution to a problem is written some memory is required to complete. For any algorithm memory may be used for the following:

1. Variables (This includes the constant values, temporary values)
2. Program instruction
3. Execution

Space complexity is the amount of memory used by the algorithm (including the input values to the algorithm) to execute and produce the result.

Sometime Auxiliary Space is confused with Space Complexity. But Auxiliary Space is the extra space or the temporary space used by the algorithm during its execution.

```
Space Complexity = Auxiliary Space + Input space
```

Memory Usage while Execution

While executing, algorithm uses memory space for three reasons:

1. **Instruction Space:** It is the amount of memory used to save the compiled version of instructions.
2. **Environmental Stack:** Sometimes an algorithm (function) may be called inside another algorithm(function). In such a situation, the current variables are pushed onto the system stack, where they wait for further execution and then the call to the inside algorithm(function) is made. For example, if a function A() calls function B() inside it, then all the variables of the function A() will get stored on the system stack temporarily, while the function B() is called and executed inside the function A().
3. **Data Space:** Amount of space used by the variables and constants. But while calculating the Space Complexity of any algorithm, we usually consider only Data Space and we neglect the Instruction Space and Environmental Stack.

Calculating the Space Complexity

For calculating the space complexity, we need to know the value of memory used by different type of datatype variables, which generally varies for different operating systems, but the method for calculating the space complexity remains the same.

Table 1.7

Type	Size
bool, char, unsigned char, signed char, __int8	1 byte
__int16, short, unsigned short, wchar_t, __wchar_t	2 bytes
float, __int32, int, unsigned int, long, unsigned long	4 bytes
double, __int64, long double, long long	8 bytes

Now let us learn how to compute space complexity by taking a few examples:

```
z = a + b + c
return z
```

In the above expression, variables a, b, c and z are all integer types, hence they will take up 4 bytes each, so total memory requirement will be (4(4) + 4) = 20 bytes, this additional 4 bytes is for return value. And because of this space requirement is fixed for the above example, hence it is called Constant Space Complexity.

Let us have another example, this time a bit complex one,

```
def sum_array(a, n):
  x = 0  # accumulator
  for i in range(n):
    x = x + a[i]
  return x
```

In the above code, 4n bytes of space is required for the array a[] elements. 4 bytes each for x, n, i and the return value. Hence the total memory requirement will be (4n + 12), which is increasing linearly with the increase in the input value n, hence it is called as Linear Space Complexity. Similarly, we can have quadratic and other complex space complexity as well, as the complexity of an algorithm increases. But we should always focus on writing algorithm code in such a way that we keep the space complexity minimum.

1.7 ASYMPTOTIC NOTATION- BIG-O, THETA AND OMEGA [Oct. 22]

Asymptotic notations are used to represent the complexities of algorithms for asymptotic analysis. These notations are mathematical tools to represent the complexities. There are three notations that are commonly used.

1. Big Oh Notation

Big-Oh (O) notation gives an upper bound for a function f(n) to within a constant factor.

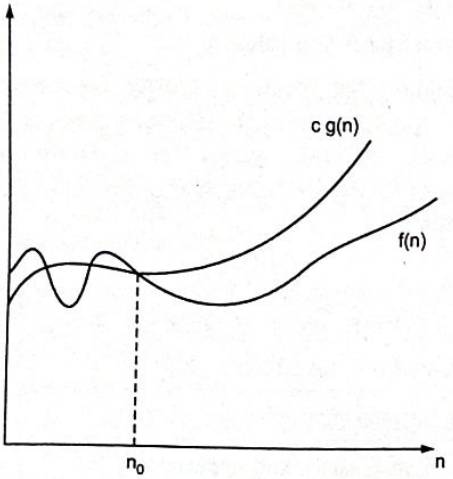


Fig. 1.6

We write f(n) = O(g(n)), if there are positive constants n_0 and c such that, to the right of n_0 the f(n) always lies on or below c g(n).

O(g(n)) = { f(n) : There exist positive constant c and n_0 such that $0 \le f(n) \le c\ g(n)$, for all $n \ge n_0$}

2. Big Omega Notation

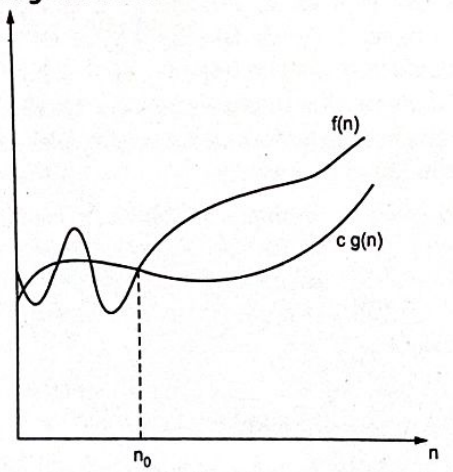


Fig. 1.7

Big-Omega (Ω) notation gives a lower bound for a function f(n) to within a constant factor.

We write f(n) = Ω(g(n)), If there are positive constants n_0 and c such that, to the right of n_0 the f(n) always lies on or above c g(n).

Ω(g(n)) = { f(n) : There exist positive constant c and n_0 such that $0 \le c\ g(n) \le f(n)$, for all $n \ge n_0$}

3. Big Theta Notation

Big-Theta(Θ) notation gives bound for a function f(n) to within a constant factor.

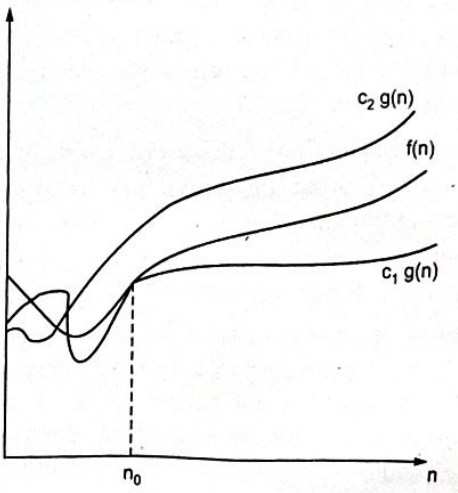


Fig. 1.8

We write f(n) = Θ(g(n)), If there are positive constants n_0 and c_1 and c_2 such that, to the right of n_0 the f(n) always lies between c_1 g(n) and c_2 g(n) inclusive.

Θ(g(n)) = {f(n) : There exist positive constant c_1, c_2 and n_0 such that $0 \le c_1\ g(n) \le f(n) \le c_2\ g(n)$, for all $n \ge n_0$}

1.8 FINDING COMPLEXITY USING STEP COUNT METHOD

The step count method is one of the methods to analyze the algorithm. In this method, we count number of times one instruction is executing.

From that we will try to find the complexity of the algorithm.

Suppose we have one algorithm to perform sequential search. Suppose each instruction will take c_1, c_2, ... amount of time to execute, then we will try to find out the time complexity of this algorithm

Algorithm	Number of times	Cost
seqSearch(arr, n, key)	1	c_1
i := 0	n+1	c_2
while i< n, do	n	c_3
if arr[i] = key, then	0/1	c_4
break		
end if		
done		
return i	1	c_5

Now if we add the cost by multiplying the number of times it is executed, (considering the worst-case situation), we will get

$$\text{Cost} = c_1 + (n+1)c_2 + nc_3 + c_4 + c_5$$
$$\text{Cost} = c_1 + nc_2 + c_2 + nc_3 + c_4 + c_5$$
$$\text{Cost} = n(c_2+c_3) + c_1 + c_4 + c_5$$
$$\text{Cost} = n(c_2+c_3) + C$$

Considering the $c_1 + c_4 + c_5$ is C, so the final equation is like straight line y = mx + b. So, we can say that the function is linear. The complexity will be O(n).

1.9 ANALYSIS OF PROGRAMMING CONSTRUCTS-LINEAR, QUADRATIC, CUBIC, LOGARITHMIC

A data structure is a systematic way of organizing and accessing data and an algorithm is a step-by-step procedure for performing some tasks in a finite amount of time. To classify some data structures and algorithms as "good", we must have precise ways of analyzing them.

Analyzing the efficiency of a program involves characterizing the running time and space usage of algorithms and data structure operations. Particularly, the running time is a natural measure of goodness, since time is precious.

Running Time

Most algorithms transform input objects into output objects. The running time of an algorithm or a data structure method typically grows with the input size, although it may also vary for different inputs of the same size. Also, the running time is affected by a lot of factors, such as the hardware environment and the software environment. In spite of the possible variations that come from different environmental factors, we would like to focus on the relationship between the running time of an algorithm and the size of its input. In the end, we would like to characterize an algorithm's running time as a function f(n) of the input size n.

But what is the proper way of measuring it?

- Experimental analysis
- Theoretical analysis

Common Functions Used in Analysis

1. The Constant Function f(n) = C

For any argument n, the constant function f(n) assigns the value C. It does not matter what the input size n is, f(n) will always be equal to the constant value C. The most fundamental constant function is f(n) = 1 and this is the typical constant function that is used in the book.

The constant function is useful in algorithm analysis, because it characterizes the number of steps needed to do a basic operation on a computer, like adding two numbers, assigning a value to some variable or comparing two numbers. Executing one instruction a fixed number of times also needs constant time only.

Constant algorithm does not depend on the input size.

Examples: Arithmetic calculation, comparison, variable declaration, assignment statement, invoking a method or function.

2. The Logarithm Function f(n) = log n

It is one of the interesting and surprising aspects of the analysis of data structures and algorithms. The general form of a logarithm function is $f(n) = \log_b n$, for some constant $b > 1$. This function is defined as follows:

$x = \log_b n$, if and only if $b^x = n$

The value b is known as the base of the logarithm. Computing the logarithm function for any integer n is not always easy, but we can easily compute the smallest integer greater than or equal to $\log_b n$, for this number is equal to the number of times we can divide n by b until we get a number less than or equal to 1. For example, $\log_3 27$ is 3, since 27/3/3/3 = 1. Likewise, $\log_2 12$ = 4, since 12/2/2/2/2 = 0.75 <= 1.

The base two approximating arises in the algorithm analysis, since a common operation in many algorithms is to repeatedly divide an input in half. In fact, the most common base for the logarithm in computer science is 2. We typically leave it off when it is 2.

Logarithm function gets slightly slower as n grows. Whenever n doubles, the running time increases by a constant.

Example: Binary search.

3. The Linear Function f(n) = n

Another simple yet important function. Given an input value n, the linear function f assigns the value n itself.

This function arises in an algorithm analysis any time we do a single basic operation for each of n elements. For example, comparing a number x to each element of an array of size n will require n comparisons. The linear function also represents the best running time we hope to achieve for any algorithm that processes a collection of n inputs.

Whenever n doubles, so does the running time.

Example: Print out the elements of an array of size n.

4. The N-Log-N Function f(n) = n log n

This function grows a little faster than the linear function and a lot slower than the quadratic function (n^2). If we can improve the running time of solving some problem from quadratic to N-Log-N, we will have an algorithm that runs much faster in general. It scales to a huge problem, since whenever n doubles, the running time more than doubles.

Example: Merge sort.

5. The Quadratic Function $f(n) = n^2$

It appears a lot in the algorithm analysis, since there are many algorithms that have nested loops, where the inner loop performs a linear number of operations and the outer loop is performed a linear number of times. In such cases, the algorithm performs $n \times n = n^2$ operations.

The quadratic function can also be used in the context of nested loops where the first iteration of a loop uses one operation, the second uses two operations, the third uses three operations and so on. That is, the number of operations is $1 + 2 + 3 + \ldots + (n-1) + n$.

For any integer $n >= 1$.

We have $1 + 2 + 3 + \ldots + (n-1) + n = n \times \frac{(n+1)}{2}$.

Quadratic algorithms are practical for relatively small problems. Whenever n doubles, the running time increases fourfold.

Example: Some manipulations of the n by n array.

6. The Cubic Function and Other Polynomials

The cubic function $f(n) = n^3$. This function appears less frequently in the context of the algorithm analysis than the constant, linear and quadratic functions. It is practical for use only on small problems. Whenever n doubles, the running time increases eightfold.

Example: $n \times n$ matrix multiplication.

The functions we have learned so far can be viewed as, all being part of a larger class of functions, the polynomials. A polynomial function is a function of the form:

$a_n x^n + a_{n-1}x^{n-1} + \ldots + a_2x^2 + a_1x + a_0$.

where $a_0, a_1, \ldots, a_n$ are constants, called the coefficients of the polynomial and $a_n ! = 0$. Integer n, which indicates the highest power in the polynomial, is called the degree of the polynomial.

7. The Exponential Function $f(n) = b^n$

In this function, b is a positive constant, called the base and the argument n is the exponent. In the algorithm analysis, the most common base for the exponential function is $b = 2$. For instance, if we have a loop that starts by performing one operation and then doubles the number of operations performed with each iteration, then the number of operations performed in the nth iteration is 2^n.

Exponential algorithm is usually not appropriate for practical use.

Example: Towers of the Hanoi.

8. The Factorial Function $f(n) = n!$

Factorial function is even worse than the exponential function. Whenever n increases by 1, the running time increases by a factor of n.

For example, permutations of n elements.

Comparing Growth Rates

Ideally, we would like data structure operations to run in times proportional to the constant or logarithm function and we would like our algorithms to run in linear or n-log-n time. Algorithms with quadratic or cubic running times are less practical, but algorithms with exponential running times are infeasible for all but the smallest sized inputs.

Let's draw the growth rates for the above functions and take a look at the following Table 1.8.

Table 1.8

	Constant	Logarithmic	Linear	N-log-N	Quadratic	Cubic	Exponential
n	O(1)	O(log n)	O(n)	O(n log n)	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4096	65536
32	1	5	32	160	1024	32768	4294967296
64	1	6	64	384	4069	262144	1.84×10^{19}

Focusing on the Worst Case

Think about the example of a linear search on an array. What are the best-, average- and worst-case performance?

An algorithm may run faster on some inputs than it does on others of the same size. Thus, we may wish to express the running time of an algorithm as the function of the input size obtained by taking the average over all possible inputs of the same size. However, an average case analysis is typically challenging.

An average case analysis usually requires that we calculate expected running times based on a given input distribution, which usually involves sophisticated probability theory. In real-time computing, the worst-case analysis is often of particular concern since it is important to know how much time might be needed in the worst case to guarantee that the algorithm would always finish on time. The term best case performance is used to describe the way an algorithm behaves under optimal conditions.

A worst-case analysis is much easier than an average case analysis, as it requires only the ability to identify the worst-case input. This approach typically leads to better algorithms. Making the standard of success for an algorithm to perform well in the worst case necessarily requires that it will do well on every input.

1.10 ALGORITHMIC STRATEGIES: INTRODUCTION TO ALGORITHM DESIGN STRATEGIES [Oct. 22]

Introduction to Algorithm Design Strategies

Algorithm design strategies are fundamental approaches to solving computational problems efficiently. Each strategy provides a general blueprint that can be adapted to solve specific problems.

Among the most important and widely used strategies are:

- Divide and Conquer
- Greedy Strategy

These strategies help in reducing the time complexity and enhancing the efficiency of algorithms.

1.10.1 Divide and Conquer Strategy [Oct. 22]

Definition:

Divide and Conquer is a top-down approach to problem-solving that works by breaking a problem into smaller subproblems, solving each subproblem independently, and then combining their results to solve the original problem.

Three Main Steps:

1. **Divide:** Split the problem into smaller subproblems that are similar to the original problem.
2. **Conquer:** Recursively solve the subproblems.
3. **Combine:** Merge the solutions of the subproblems to get the final solution.

Characteristics:

- Problems must be recursively reducible.
- Subproblems must be independent of each other.
- Combining solutions should be efficient.

Common Examples:

- **Merge Sort:** Divides array into halves, sorts them recursively, and merges the sorted halves.
- **Quick Sort:** Chooses a pivot, partitions the array, and recursively sorts partitions.
- **Binary Search:** Repeatedly halves the search space in a sorted array.
- **Strassen's Matrix Multiplication:** Reduces the number of multiplication operations for large matrices.

Advantages

- Breaks down complex problems into manageable subproblems.
- Naturally fits recursive implementations.
- Efficient in solving large datasets with logarithmic depth.

Disadvantages

- May involve repeated computation of the same subproblems (can be mitigated with memorization).
- Recursion overhead in terms of stack space and function calls.

1.10.2 Greedy Strategy [Oct. 22]

Definition:

Greedy Strategy is a problem-solving approach that builds up a solution piece by piece, always choosing the next piece that offers the most immediate benefit or the "best" local option, with the hope that this approach leads to an optimal global solution.

Steps in Greedy Approach:

1. **Greedy Choice Property:** Make the best possible choice at each step.
2. **Feasibility Check:** Ensure the choice does not violate constraints.
3. **Solution Assembly:** Construct the final solution based on the greedy choices.

Characteristics:

- Makes decisions based on current local optimum.
- Does not revise previous decisions (non-backtracking).
- Often faster and simpler than other approaches.

Common Examples:

- **Fractional Knapsack Problem:** Select items with the highest value/weight ratio.
- **Activity Selection Problem:** Select the maximum number of non-overlapping activities.
- **Huffman Coding:** Build an optimal prefix code based on frequency of characters.
- **Dijkstra's Algorithm:** Finds the shortest path from a source node to all others.

Advantages:

- Simplicity and ease of implementation.
- Faster in practice for many problems.
- Uses less memory compared to recursive approaches.

Disadvantages:

- Not all problems can be solved optimally using greedy strategy.
- Requires careful analysis to ensure correctness.
- May lead to suboptimal solutions if the problem lacks the greedy-choice property or optimal substructure.

Table 1.9: Comparison: Divide and Conquer vs. Greedy

Feature	Divide and Conquer	Greedy Strategy
Approach	Top-down, recursive	Iterative, builds solution step-by-step
Subproblem Relation	Independent	Decisions made once, no backtracking
Optimal Substructure	Required	Required

Feature	Divide and Conquer	Greedy Strategy
Common Use Cases	Sorting, searching, matrix operations	Optimization problems
Complexity	Often logarithmic (e.g., O(n log n))	Usually linear or polynomial
Example Problems	Merge Sort, Quick Sort, Binary Search	Knapsack, Activity Selection, Dijkstra

CASE STUDY

1. E-commerce Product Sorting using Divide and Conquer Strategy

Problem Statement

In an e-commerce platform, users expect rapid sorting of thousands of product listings by criteria like price, rating, or discount. Sorting must be both time-efficient and stable to ensure user satisfaction and correct order display.

Real-World Context

E-commerce websites like Amazon, Flipkart, or Myntra often need to sort large catalogs of products based on multiple criteria. When a user selects "Sort by Price (Low to High)," the backend should sort potentially thousands of items in real-time without delays.

This is a perfect use case for the Merge Sort algorithm, a classic Divide and Conquer strategy that ensures O(n log n) performance, ideal for large-scale data.

Data Structures Used

- **Array:** Used to store the product data.
- **Merge Sort:** Recursive algorithm that divides the array and merges sorted subarrays.

Why These Structures?

- **Array** provides fast index-based access.
- **Merge Sort** is a stable sort (maintains relative order of equal elements) and offers better worst-case performance than quicksort.

Algorithm Explanation

Merge sort works as follows:

1. Divide the array into two halves.
2. Recursively sort each half.
3. Merge the sorted halves into a single sorted array.

This approach ensures each step handles a smaller, manageable chunk, reducing the complexity.

```
from dataclasses import dataclass
from typing import List

@dataclass
class Product:
    name: str
    price: float
    rating: float
# Merge two sorted halves

def merge(arr: List[Product], left: int, mid: int, right: int, key: str):
    L = arr[left:mid + 1]
    R = arr[mid + 1:right + 1]
    i = j = 0
    k = left
    while i < len(L) and j < len(R):
        if key == "price":
            condition = L[i].price <= R[j].price
        else:  # rating
            condition = L[i].rating >= R[j].rating
        if condition:
            arr[k] = L[i]
            i += 1
        else:
            arr[k] = R[j]
            j += 1
        k += 1
    while i < len(L):
        arr[k] = L[i]
        i += 1
        k += 1
    while j < len(R):
        arr[k] = R[j]
        j += 1
        k += 1

# Recursive merge sort
def merge_sort(arr: List[Product], left: int, right: int, key: str):
    if left < right:
        mid = (left + right) // 2
        merge_sort(arr, left, mid, key)
        merge_sort(arr, mid + 1, right, key)
        merge(arr, left, mid, right, key)
# Display product list
def display(arr: List[Product]):

    for p in arr:
        print(f"{p.name} - Rs. {p.price} - Rating: {p.rating}")
# Main execution
```

```
products = [
    Product("Laptop", 75000, 4.2),
    Product("Smartphone", 30000, 4.5),
    Product("Tablet", 20000, 4.0),
    Product("Headphones", 5000, 4.8),
    Product("Monitor", 12000, 4.3)
]

print("\nSorted by Price:")
merge_sort(products, 0, len(products) - 1, "price")
display(products)

print("\nSorted by Rating:")
merge_sort(products, 0, len(products) - 1, "rating")
display(products)
```

Complexity Analysis

- **Time Complexity:** O(n log n)
- **Space Complexity**: O(n) auxiliary space
- **Stable Sort:** Yes

Enhancements

- Multi-key sorting (e.g., price, then rating)
- Parallel processing using threads

Takeaways

Merge Sort ensures reliable, consistent performance on large product datasets. Its stability makes it suitable for ecommerce systems where multiple products may share the same price or rating.

2. Google Calendar Application Using Greedy Strategy

Title

Meeting Slot Optimization in Google Calendar using Greedy Strategy

Problem Statement

How can Google Calendar automatically schedule the maximum number of non-overlapping meetings given many user requests throughout the day?

Real-World Context

Users may request overlapping time slots for meetings. The system must allocate time efficiently to accommodate the highest number of meetings. This classic Activity Selection Problem is solved optimally using a Greedy Algorithm.

Data Structures Used

- **Vector of Pairs:** Stores meeting intervals (start, end)
- **Sorting:** By meeting end time

Why These Structures?

- Intervals naturally represent time slots.
- Sorting by end time helps select meetings that leave the most room for others.

Algorithm Explanation

1. Sort meetings by end time.
2. Pick the meeting that finishes earliest.
3. Skip overlapping meetings.
4. Repeat for remaining non-overlapping meetings.

```
from typing import List, Tuple
Meeting = Tuple[float, float]

def schedule_meetings(meetings: List[Meeting]):
    # Sort by end time
    meetings.sort(key=lambda x: x[1])
    last_end = 0

    print("Scheduled Meetings:")
    for start, end in meetings:
        if start >= last_end:

            print(f"Start: {start} End: {end}")
            last_end = end

# Main execution
meetings = [
    (9.0, 10.0),
    (9.5, 11.0),
    (10.0, 10.5),
    (11.0, 12.0),
    (13.0, 14.0),
    (12.5, 13.5)
]
schedule_meetings(meetings)
```

Complexity Analysis

- **Time Complexity:** O(n log n) for sorting
- **Space Complexity:** O(1) (if sorting in-place)

Enhancements

- Handle weighted meetings (importance factor).
- Multi-room scheduling using heaps.

Takeaways

Greedy algorithms are ideal for problems requiring maximum efficiency in resource allocation. Google Calendar-like systems benefit from their speed and optimality in slot utilization.

EXERCISE

1. Define Abstract Data Types (ADT). How is it different from Data Structures? Give two examples.
2. Differentiate between Linear and Non-linear data structures with at least two examples for each.
3. Explain the terms Static and Dynamic data structures. Give one practical use case for each.
4. What do you mean by Persistent and Ephemeral data structures? How do persistent structures retain history?
5. Define the following with respect to algorithm analysis:
   (a) Space Complexity
   (b) Time Complexity

   Also, mention why analyzing both is important in algorithm design.
6. Explain the difference between Big-O, Theta (Θ), and Omega (Ω) notations with one example each.
7. Describe the Divide and Conquer approach. Apply it to explain how Merge Sort works with an example of array [8, 4, 2, 6].
8. What is the Greedy strategy in algorithm design? List two problems where the Greedy method gives an optimal solution and briefly justify why it works in each.
9. Classify the following algorithms according to their time complexity:
   (a) Linear Search
   (b) Bubble Sort
   (c) Merge Sort
   (d) Binary Search
   (e) Matrix Multiplication (naïve)
10. Analyze the time complexity of the following code using the step-count method:

```
n = 5  # Example value, change as needed
for i in range(n):
  for j in range(n):
    print(f"{i},{j}")
```

State the exact number of steps and the final time complexity class.

✠ ✠ ✠